Technical Report

worldcup-fantasy

Design of the data pipeline and the decision engine

FIFA Fantasy, World Cup 2026

Simone Paoletti

Contents

1 Introduction **4**

1.1 Scope of this document . . . 4

1.2 Two constraints that shaped the design . . . 4

1.3 Why the price drives three things at once . . . 5

2 Architecture **6**

2.1 The pipeline . . . 6

2.2 Module map . . . 6

2.3 The clean-degrade contract . . . 7

3 Data collected **9**

3.1 FIFA fantasy feeds . . . 9

3.1.1 Checksum caching and snapshots . . . 9

3.1.2 Verified feed facts . . . 10

3.2 FIFA Match Centre . . . 10

3.2.1 Three signals . . . 10

3.2.2 The team and player bridge . . . 11

3.2.3 Recency-weighted start probability . . . 11

3.2.4 Deriving suspensions from bookings . . . 11

3.2.5 The lineup-less cache poisoning . . . 11

3.3 The Odds API . . . 11

3.3.1 Two markets . . . 12

3.3.2 Picking the right sport key . . . 12

3.3.3 Bridging team names . . . 12

3.4 Historical internationals . . . 12

3.5 Economics . . . 12

3.6 Club season statistics . . . 13

3.6.1 FBref column limits . . . 13

3.6.2 The double-count trap . . . 13

3.6.3 Name matching and its cache . . . 13

3.7 The availability aggregator . . . 13
3.7.1 Wikipedia withdrawals and the "replaced by" trap . . . 14
3.7.2 The API-Football free-plan block . . . 14
3.7.3 Shared name bridges . . . 14

4 Game rules and scoring **15**
4.1 Budget and squad . . . 15
4.2 Per-nation cap by stage . . . 15
4.3 Transfer windows . . . 15
4.4 Captain, vice and substitutions . . . 16
4.5 Boosters . . . 16
4.6 Scoring table . . . 17

5 Models **18**
5.1 Elo ratings and the outcome model . . . 18
5.1.1 Elo update . . . 18
5.1.2 Outcome model . . . 18
5.2 Monte Carlo progression . . . 19
5.2.1 The simulation . . . 19
5.2.2 Market calibration . . . 19
5.2.3 Two horizon outputs . . . 19
5.3 Expected points per player . . . 19
5.3.1 The club projection . . . 20
5.3.2 The club-price blend . . . 20
5.3.3 Two separate P(plays) factors . . . 20
5.3.4 Form blend, opponent factor and scouting EV . . . 20
5.4 Calibration . . . 21
5.4.1 The observed target and the fits . . . 21
5.4.2 The improvement gate . . . 21

6 Optimization **22**
6.1 Squad selection as an ILP . . . 22
6.1.1 Objective . . . 22
6.1.2 Constraints . . . 22
6.1.3 Bench weight and the backup goalkeeper . . . 23
6.2 Transfers . . . 23
6.2.1 The paid-transfer rule . . . 23
6.2.2 Forced versus voluntary . . . 23
6.3 Captain, vice and bench order . . . 23

6.4 Option value: the timing decisions . . . 24
6.4.1 Option-value captaincy . . . 24
6.4.2 Temporal option value of the eleven . . . 25

7 Orchestration **26**
7.1 advise . . . 26
7.1.1 Picking the round to plan . . . 26
7.1.2 Apply-aware planning . . . 26
7.1.3 Assembling the move list . . . 27
7.1.4 The from-scratch comparison . . . 27
7.2 live . . . 27
7.2.1 The first-change cost . . . 27
7.2.2 Captain before he has played . . . 27
7.2.3 The round outlook . . . 27
7.2.4 Recording an executed change . . . 28
7.2.5 The apply-over-live guard . . . 28

8 State and the semi-automatic flow **29**
8.1 The two files . . . 29
8.2 The commands that write state . . . 29
8.3 Why state lives locally . . . 30

A Lessons learned **31**
A.1 FIFA feeds . . . 31
A.2 FIFA Match Centre . . . 31
A.3 The Odds API . . . 31
A.4 Club stats (FBref) . . . 32
A.5 Availability . . . 32
A.6 Models and optimizer . . . 32
A.7 Calibration . . . 33

Chapter 1

Introduction

1.1 Scope of this document

This is a technical record of the design choices behind the `worldcup-fantasy` decision engine: how it collects data and how it turns that data into the round-by-round recommendations. It documents the reasoning and the trade-offs behind each choice. It is not a user guide, and it does not cover the user-facing interfaces: the web GUI and its FastAPI layer, and the test suite, sit outside this scope. The engine is reached through a CLI and a local web GUI, and everything here describes the engine.

The chapters follow the data as it becomes a decision: the sources (Chapter 3), the rules that constrain every choice (Chapter 4), the models that produce expected points and progression probabilities (Chapter 5), the optimizer that selects the squad and the live timing (Chapter 6), the orchestration that sequences them (Chapter 7), and the local state that carries the loop between runs (Chapter 8).

1.2 Two constraints that shaped the design

Two hard constraints drove most of the engineering.

Every decision uses data current to the last played match. This is the project's governing requirement, and the code enforces it rather than assuming it. `compute_advice` refreshes the FIFA feeds before it computes anything, and the refresh is part of the flow, not an option (`advise.py:379`). The progression model folds each completed 2026 result into its Elo ratings from the live feed rather than from a static dataset (`progression.py:118`), and the bookmaker odds are re-fetched every run (`odds.py:1`). The checksum cache of Section 3.1 keeps that mandatory refresh cheap.

The engine never writes to the FIFA account. The game's write API is authenticated and out of scope, so the engine produces an instruction list and the user applies it by hand. The consequence runs deep into the design: the squad the game would hold has to be mirrored in a local file, and the engine carries guards to keep that mirror honest across schema bumps, round transitions and in-game edits. Chapter 8 covers the state model and those guards.

1.3 Why the price drives three things at once

A choice that recurs through the models and the optimizer: the player's price is fixed for the whole tournament, so it is treated not as a market quote but as a fixed expert judgement used in three distinct roles. It is half of the quality prior, blended with the club projection. It ranks starters above backups within each (squad, position) pair, a depth chart that becomes a starter probability. And it is the cost in the budget constraint. Its weight as a quality signal is not constant: it decays as observed World Cup form accumulates, with full trust in form after four matches (Section 5.3).

The second deliberate bias is the horizon. A player is scored on his next round plus the discounted expected remaining rounds of his national team, an output of the Monte Carlo progression model, so the optimizer commits to likely finalists before they are obvious picks (Section 6.1). The rest of this document is the detail behind these choices, source by source and model by model.

Chapter 2

Architecture

2.1 The pipeline

A single run of `advise` moves through four stages: fetch the data, predict expected points and probabilities, optimize the squad and decisions, report the move list. Figure 2.1 shows the stages and what happens inside each.

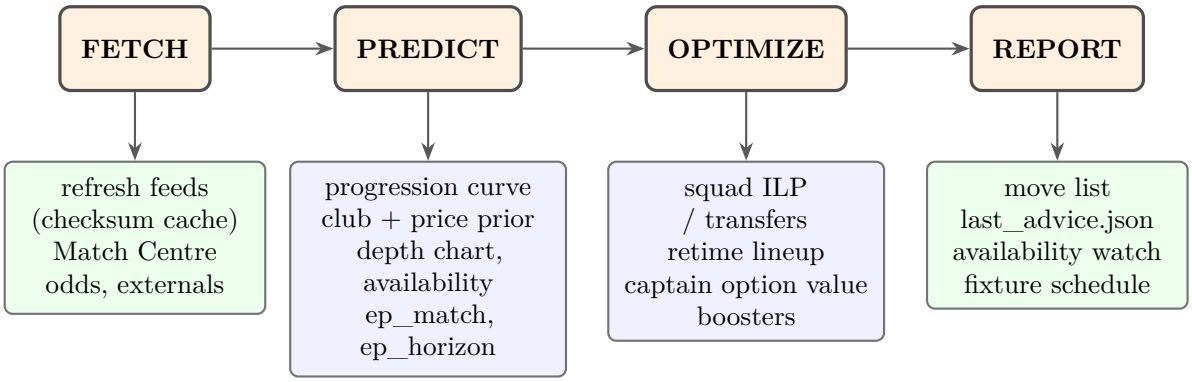


Figure 2.1: The `advise` pipeline. Fetch refreshes every source; predict builds the per-player expected points; optimize solves the squad and the live options; report prints the moves and writes `last_advice.json`.

`compute_advice` in `advise.py` is the spine. It refreshes the feeds, applies any calibration, loads the players, rounds and squads tables, runs `player_points.expected_value` to attach the expected-points columns, then calls the optimizer and the captain or lineup logic before assembling the structured `AdvicePlan` (`advise.py:363`). The intra-round command `live` follows the same fetch-then-decide shape against partial results (`live.py:636`).

2.2 Module map

Figure 2.2 groups the modules into the four responsibilities they serve. Data sources return normalized tables and dictionaries; models turn those into expected points and survival probabilities; the optimizer turns those into an integer program and a set of live-timing decisions; the orchestrator drives the sequence and renders the result.

Table 2.1 lists the modules and their jobs.

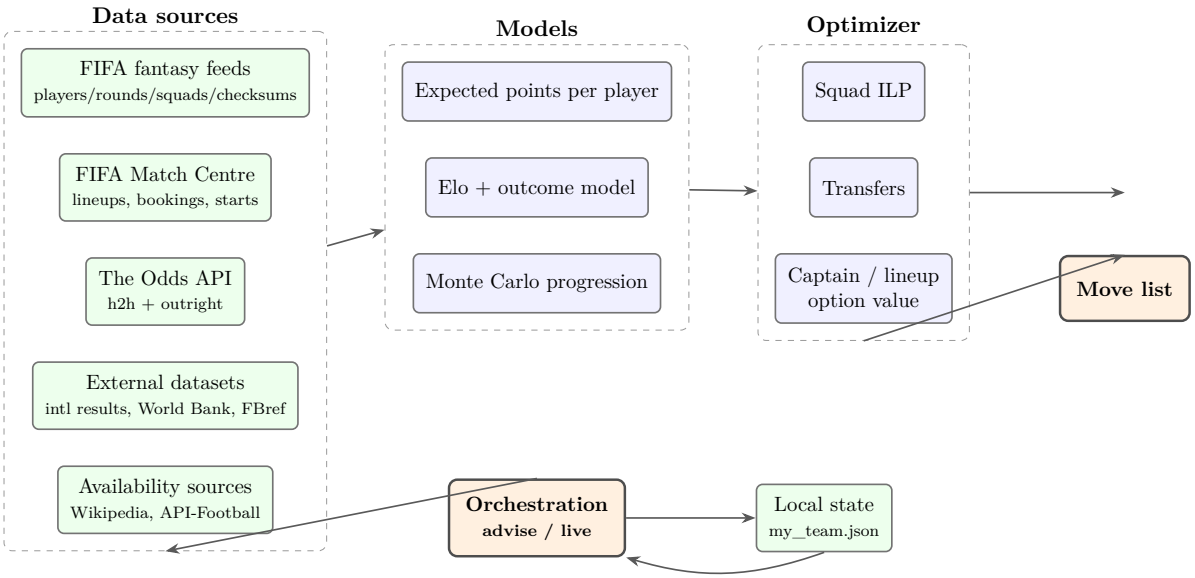


Figure 2.2: Module map of the decision engine. Data sources feed the models, the models feed the optimizer, and the orchestrator (`advise` / `live`) drives the sequence and reads or writes the local state.

2.3 The clean-degrade contract

Every optional data source returns an empty result on any failure: no key, a network error, a rate limit, a schema change. The aggregator and the models treat the empty result as "no signal here" and fall back to what they already have. The Odds API returns `None` without a key and the model runs on Elo alone (`odds.py:8`). The FIFA Match Centre helpers return `{}` on any failure so availability degrades to the price depth chart (`fifa_match.py:23`). The progression simulation wraps its whole body and falls back to a price heuristic if a source breaks, with a `WCFANTASY_STRICT` environment switch to re-raise during debugging (`progression.py:173`). One broken source never takes the tool down; it quietly removes one input.

Table 2.1: Decision-engine modules and responsibilities.

Module	**Responsibility**
`config.py`	Paths, feed URLs, the men's World Cup competition and season ids, HTTP headers.
`rules.py`	Budget, squad shape, per-nation caps, transfer windows, suspensions, boosters, the full scoring table.
`fifa_client.py`	Feed download with checksum caching and timestamped snapshots; the single HTTP path to FIFA.
`feed_status.py`	Canonical status vocabularies (done / live / pending) in one place.
`fixtures.py`	Kickoff parsing and `squadId`-to-kickoff maps, all timezone aware.
`features.py`	The player table that models and optimizer consume.
`data_sources/`	FIFA Match Centre, The Odds API, historical results, economics, club stats, availability aggregator, name bridges.
`models/`	Elo and the outcome model, Monte Carlo progression, expected points per player.
`optimizer/`	Squad ILP, transfer allocation, captain/vice/bench and the option-value timing.
`advise.py`	Orchestrator for the between-rounds plan; builds `AdvicePlan`.
`live.py`	Intra-round captain and substitution advice; records executed changes.
`state.py`	The local squad record and its load/save.
`calibrate.py`	Offline refit of the priors against real results, gated on the metrics.

Chapter 3

Data collected

This chapter is the heart of the document. It covers every source the advisor reads, what it pulls from each, how it bridges that data to fantasy players, and the constraints learned the hard way. Each source follows the same contract: it returns an empty result on failure, and the model falls back.

3.1 FIFA fantasy feeds

The base is `https://play.fifa.com/json/fantasy/`. The CDN answers 403 without a browser-like `User-Agent`, so every request carries one (`config.py:11`). Four feeds matter, listed in Table 3.1.

Table 3.1: Public FIFA fantasy feeds.

Feed	Content
`players.json`	About 1500 players. `price` (the fixed fantasy value), `position` (GK/DEF/MID/FWD), `squadId`, `status`, `percentSelected`, and a `stats` block with `totalPoints`, `avgPoints`, `form` and `roundPoints` per round.
`rounds.json`	Eight rounds with every fixture: dates, venues, `homeSquadId` / `awaySquadId`, scores and scorers, filled in live during the tournament.
`squads.json`	The 48 national teams: id, name, group, `abbr` (the FIFA 3-letter code) and `isEliminated`.
`checksums.json`	MD5 of the `players` and `rounds` feeds, so a feed is re-downloaded only when its hash changed.

3.1.1 Checksum caching and snapshots

`refresh()` fetches `checksums.json`, compares each remote hash to the locally stored one, and downloads only the feeds that changed. A changed feed is written to `data/raw/latest/` and also snapshotted with a UTC timestamp under `data/raw/<feed>/`, building the historical dataset of prices, ownership and round points the models can mine later (`fifa_client.py:68`). The `squads` feed has no checksum entry, so it is treated as always-fetch.

```
known = local_sums.get(feed)
remote = remote_sums.get(feed)  # squads has no checksum entry
needs = force or remote is None or known != remote or not _latest_path(feed).exists()
if not needs:
    changed[feed] = False
```

```
    continue
data = _fetch(feed)
_latest_path(feed).write_bytes(data)
```

Listing 3.1: Checksum-gated refresh (excerpt).

The `player_table` builder flattens a player record into the table the rest of the engine uses, deriving `matchesPlayed` from the length of the `roundPoints` block and joining the squad name, group and elimination flag (`features.py:6`).

3.1.2 Verified feed facts

Reading the feed at matchday 1 (2026-06-12) settled three points the schema does not document. Appearance points are cumulative: a player who reaches sixty minutes carries 2 points, a short cameo carries 1. The status strings are `complete` / `playing` / `scheduled`, not `completed`. And `roundPoints` is a dict keyed by the round number as a string, for example `{'1': 13}`. The status vocabularies live in `feed_status.py` so a later schema change is a one-file edit.

3.2 FIFA Match Centre

The fantasy feed only exposes data observed at full time. The official Match Centre feed at `https://api.fifa.com/api/v3` carries the pre-match signals the fantasy feed lacks, free and key-less with the same browser User-Agent. The men's World Cup is competition `17`; the 2026 tournament is season `285023` (both verified against the live feed). Two endpoints matter, in Table 3.2.

Table 3.2: FIFA Match Centre endpoints.

Endpoint	Content
`/calendar/matches`	All 104 fixtures: `IdStage`, `IdMatch`, home and away `IdCountry`, `MatchStatus` (0 completed, 1 scheduled).
`/live/football/17/285023/{stage}/{match}`	Match detail with `Players` (Status 1 starting XI, 2 bench, populated about an hour before kickoff) and `Bookings` (Card 1 yellow, 2 sending-off).

3.2.1 Three signals

`fifa_match.py` derives three signals from this feed.

Confirmed lineups (`lineup_factors`). Published roughly an hour before kickoff, the strongest availability signal and used mainly in `live`. A named starter scores 1.0, a named substitute 0.5, and a squad member who is on neither list scores 0.0, but only when every published name bridged to a fantasy player (`fifa_match.py:186`).

Suspensions (`suspension_factors`). Derived from the `Bookings` of completed matches through the disciplinary rules: a sending-off, or a second accumulated yellow, bans the next match.

Observed starts (`observed_start_factors`). Who actually took the pitch in completed matches, turned into a recency-weighted start probability. This is the data-driven depth chart that supersedes the price rank, which had ranked the wrong keeper for Spain and Egypt at matchday 1.

3.2.2 The team and player bridge

The team bridge is exact: the fantasy `abbr` equals the Match Centre `IdCountry`, both FIFA 3-letter codes, so no fuzzy team matching is needed (`fifa_match.py:164`). Inside a matched squad the player bridge is fuzzy on the name, which correctly rejects a wrong same-surname match: the feed's "Sphephelo Sithole" is not confused with "Yaya Sithole".

3.2.3 Recency-weighted start probability

A more recent match weighs more, with a geometric decay per round back. The estimate is clamped to a floor and a ceiling so one start is strong evidence but never a certainty, since managers rotate (`fifa_match.py:281`).

```
ordered = sorted(observations, key=lambda t: t[0], reverse=True)
num = den = 0.0
for i, (_, started) in enumerate(ordered):
    w = OBSERVED_RECENCY_DECAY**i
    num += w * (1.0 if started else 0.0)
    den += w
return min(OBSERVED_START_CEILING, max(OBSERVED_START_FLOOR, num / den))
```

Listing 3.2: Recency-weighted start rate.

3.2.4 Deriving suspensions from bookings

`classify_bookings` splits a match's cards into single yellows and sending-offs, collapsing two yellows for the same player in one match into a sending-off. `derive_suspensions` then walks each team's completed matches in round order: a sending-off or the second accumulated yellow sets a pending ban and clears the tally; a pending ban clears once the player is absent from a later matchday squad (the ban was served); and the single-yellow tally resets after the group stage and again after the quarter-finals, matching the official card-wipe rule (`fifa_match.py:358`). A ban still pending after the last completed match applies next round.

3.2.5 The lineup-less cache poisoning

A completed match is immutable, so its detail is cached on disk. A match detail fetched before the lineup is published carries empty `Players` on both sides. Caching that empty payload for a since-completed match poisons every later read. The fix: only cache a payload that carries a lineup, and treat a cached but lineup-less payload as a miss and refetch it. This both prevents the poisoning and self-heals files already poisoned by the earlier always-cache behaviour (`fifa_match.py:126`).

3.3 The Odds API

Bookmaker odds come from The Odds API and are fetched fresh on every run, about two requests against a 500-per-month free quota, so the prices reflect lineups and news up to launch time (`odds.py:1`). The key is read from `ODDS_API_KEY` or `odds_api_key.txt`; without a key everything returns `None` and the model runs on Elo alone.

3.3.1 Two markets

The head-to-head market pins per-fixture probabilities. Implied probabilities are de-overrounded per bookmaker by dividing each inverse price by the book's total, then averaged across bookmakers and regions (`odds.py:107`). The outright winner market calibrates the ratings through a second simulation pass.

```
inv = {name: 1.0 / p for name, p in prices.items() if p and p > 1.0}
total = sum(inv.values())
return {name: v / total for name, v in inv.items()} if total else {}
```

Listing 3.3: De-overrounding one bookmaker's prices.

3.3.2 Picking the right sport key

The sport key must be the men's national World Cup, not anything that merely contains `world_cup`: the cricket and rugby world cups, the FIFA Club World Cup, the women's tournament and the qualifiers all share that substring. The cricket key was being picked for the head-to-head market, which then returned zero soccer fixtures. The selector restricts to the `soccer_fifa_world_cup` family, which covers both the match and the `_winner` markets, and drops the women's and qualifier variants (`odds.py:64`).

3.3.3 Bridging team names

The market spells some teams differently from FIFA. `match_team` tries the shared override map (`names.py`), then the historical-dataset spelling, then a high-cutoff fuzzy match, and finally gives up quietly: an unmatched name is expected for a non-qualified team in the outright market (`odds.py:90`).

3.4 Historical internationals

The Elo ratings are built from about 49,000 historical international results, the martj42 dataset (the same data as the Kaggle "International football results from 1872"), cached and re-downloaded when older than twelve hours (`intl_results.py:1`). The loader drops unplayed fixtures and any World Cup 2026 rows: those matches are applied from the live FIFA feed instead, which both updates faster and avoids double-counting. A small override table maps the eight FIFA names that differ from the dataset spelling.

3.5 Economics

World Bank GDP per capita and population enter as a small Elo adjustment, in the spirit of Joachim Klement's World Cup models that blend wealth and population (a proxy for talent pool and infrastructure) with football data (`economics.py:1`). The output is z-scored log GDP per capita and log population, weighted at 20 and 10 Elo points per standard deviation, kept deliberately small so the market and the football data drive the model and the economics only nudge it. The data is cached for seven days and a failed download falls back to the cached copy.

3.6 Club season statistics

The fantasy feed carries no pre-tournament quality signal beyond price and ownership. The 25/26 club season provides one: per-player season stats from FBref via the optional `soccerdata` dependency, normalized into the flat schema in Table 3.3 and cached as a CSV (`club_stats.py:1`).

Table 3.3: Normalized club-stats schema (season totals per player).

`name`, `nation`, `position` (GK/DEF/MID/FWD), `games`, `minutes`, `goals`, `assists`, `shots_on_target`, `tackles`, `chances_created`, `saves`, `clean_sheets`, `goals_conceded`, `yellow`, `red`, `penalties_won`, `penalties_conceded`

3.6.1 FBref column limits

This `soccerdata` version exposes only the `standard`, `keeper`, `shooting`, `playing_time` and `misc` player season tables, with no `passing` or `defense`, and the 25/26 Big-5 pages carry no expected-goals columns. Two proxies follow from that: tackles come from the `misc` table's tackles-won, and `chances_created` is estimated from assists at roughly seven key passes per assist (`club_stats.py:58`). The first fetch downloads a chromedriver to clear Cloudflare and takes minutes; pages are cached afterward.

3.6.2 The double-count trap

A player who moved clubs mid-season appears once per club. Their rows are collapsed to one before the player-summed extra tables are merged in, or the totals from the extra tables would be counted twice (`club_stats.py:128`).

3.6.3 Name matching and its cache

`match_name` bridges a FIFA name to an FBref name: token-set containment first (so "L. Messi" matches "Lionel Messi"), then a similarity ratio over accent-stripped lowercased strings, with a manual override file for the cases the fuzzy match gets wrong (`club_stats.py:187`). The full player-to-club mapping is the slowest step in an `advise` run, an $O(\text{players} \times \text{names})$ fuzzy pass, so its result is cached keyed by a hash of the inputs; an unchanged feed reuses the mapping instead of recomputing it, which cut a warm `advise` run from about 44 seconds to about 10 (`club_stats.py:208`).

3.7 The availability aggregator

The fantasy feed flags only `playing` or `transferred`: nothing about fitness, suspensions or who actually starts. `availability.py` fills that gap by merging the free signals, lowest factor wins, so any source that says a player is less likely to play takes over (`availability.py:1`).

```
def _merge(*sources: dict[int, float]) -> dict[int, float]:
    out: dict[int, float] = {}
    for source in sources:
        for pid, value in source.items():
            out[pid] = min(out.get(pid, 1.0), value)
    return out
```

Listing 3.4: Lowest-factor-wins merge.

The merged sources, strongest first:

1. **Confirmed lineups** from the Match Centre. Decisive but only close to kickoff, so kept in a separate `live_lineup_factors` call that overrides the rest in `live`.
2. **Suspensions** from FIFA bookings (Section 3.2).
3. **Injury withdrawals** from Wikipedia (below), the only free fitness signal.
4. **API-Football injuries**, optional and inert on the free plan.

3.7.1 Wikipedia withdrawals and the "replaced by" trap

The "2026 FIFA World Cup squads" article notes injury withdrawals in prose, read through the MediaWiki API; the content is CC BY-SA and programmatic access is permitted (`wikipedia_squads.py:1`). Wikitext is unstructured, so the parser is deliberately conservative. It flags only the player named immediately before a withdrawal verb with an injury noun in the same clause, and never the healthy replacement. A sentence like "[[Injured]] withdrew injured and was replaced by [[Healthy]]" must flag only the first player; an earlier version that triggered on "replaced" produced 37 false positives. A defensive cap discards the whole pass if it matches more players than a real tournament ever would, on the assumption the page format drifted (`wikipedia_squads.py:39`). Verified on 2026-06-13: 18 genuine withdrawals, zero false flags.

3.7.2 The API-Football free-plan block

API-Football injuries were the original plan, but the free plan has no access to season 2026: `/injuries` returns zero with a plan error. The path stays in the codebase, behind a key read from `API_FOOTBALL_KEY`, inert until a paid key is configured; its pure core `build_injury_factors` is still tested (`availability.py:118`). With it contributing nothing, the FIFA and Wikipedia signals carry the load.

3.7.3 Shared name bridges

The bookmaker feed and API-Football spell some national teams differently from FIFA, and the mapping is identical for both, so it lives once in `EXTERNAL_TO_FIFA` (`names.py:13`). The historical dataset maps the other direction and keeps its own table.

Chapter 4

Game rules and scoring

`rules.py` encodes the official "How to Play" rules as constants and a few helpers. The models and the optimizer read from here, so a rule change is a single edit. Rounds are numbered 1 to 8: matchdays 1 to 3, then R32, R16, quarter-finals, semi-finals and the final.

4.1 Budget and squad

The group-stage budget is $100m. When round 3 locks and the R32 transfer window opens, the budget rises by $5m automatically (`rules.py:9`). Prices are fixed for the whole tournament: the price is the experts' pre-tournament judgement of scoring potential, never a market signal, so there is no buy-low play.

A squad is fifteen players: two goalkeepers, five defenders, five midfielders, three forwards. The starting eleven must match one of the seven official formations, with the default 4-4-2 (`rules.py:31`).

4.2 Per-nation cap by stage

The number of players allowed from one country grows with the stage, in Table 4.1. `max_per_squad(round)` returns it, clamped to the 1 to 8 range (`rules.py:22`).

Table 4.1: Maximum players from one country, by round.

Round	1–3 (groups)	4 (R32)	5 (R16)	6 (QF)	7 (SF)	8 (Final)
Cap	3	3	4	5	6	8

4.3 Transfer windows

The free allocation per window is in Table 4.2. Round 1 is the pre-tournament build, with unlimited edits until the first lockout; the R32 window is also unlimited, a de-facto free rebuild. In the group stage, one unused transfer carries over, and only into round 3. Going over the allocation is allowed at a 3-point penalty per extra transfer, deducted at lockout. A transfer confirmed during a live round only applies from the next round (`rules.py:54`).

Table 4.2: Free transfers per window (R1 and R32 unlimited).

Round	1	2	3	4 (R32)	5	6	7	8
Free	∞	2	2	∞	4	4	5	6

4.4 Captain, vice and substitutions

The captain scores double. The vice-captain scores double only if the captain plays no minute and no manual change was made during the live round. The captain is switchable during a round, from a player whose match is over to one not yet played, any number of times. Bench substitutions are manual during a live round (a completed starter for a not-yet-started bench player, formation permitting) or automatic at round end for did-not-play starters, the latter only if no manual change was made that round. The first manual change of any kind cancels both auto-subs and the vice insurance for that round (`rules.py:37`).

4.5 Boosters

One booster per round at most, each usable once in the tournament. All are reversible before lockout except the Wildcard. Table 4.3 lists them.

Table 4.3: Boosters (`rules.BOOSTERS`).

Booster	**Effect**
Maximum Captain	Doubles the highest-scoring starter, captain assigned in hindsight.
12th Man	One extra scorer for the round from outside the squad, no budget or country limit; cannot be subbed, captained or transferred.
Wildcard	Unlimited transfers for one round; banned at matchday 1 and the R32; irreversible once confirmed.
Qualification	Knockout only: +2 to each starter whose team advances (needs one minute played; the captain's +2 is not doubled).
Clean Sheet Shield	Knockout only: GK, DEF and MID keep the clean-sheet bonus until the team concedes a second goal. One knockout round, final included.

The Clean Sheet Shield was the hidden "Mystery" booster, revealed when the R32 opened. A legacy alias maps a recorded or typed `mystery` onto `clean_sheet_shield` so old state still resolves (`rules.py:140`). Its effect lifts into the scoring through a `shield` flag, which raises the goals-allowed threshold from zero to one:

```
def clean_sheet_points(position, goals_conceded, *, shield=False):
    allowed = CLEAN_SHEET_SHIELD_GOALS_ALLOWED if shield else 0
    return CLEAN_SHEET_POINTS.get(position, 0) if goals_conceded <= allowed else 0
```

Listing 4.1: Clean-sheet points with and without the shield.

The user's booster strategy is encoded in the advice layer (Chapter 6): nothing in round 1, the Wildcard late in the knockouts once several squad players are eliminated, the Qualification booster held for the semi-final.

4.6 Scoring table

Table 4.4 is the full scoring table. Appearance points are cumulative: 1 for any minutes, 1 more at sixty minutes. A clean sheet requires sixty minutes. For goalkeepers and defenders the first goal conceded is free and each additional one is −1.

Table 4.4: Official scoring table (`rules.py`).

Event	**Points**
Appearance	+1 any minutes, +1 more at 60+
Goal	GK 9, DEF 7, MID 6, FWD 5
Assist	+3
Clean sheet (60+ min)	GK 5, DEF 5, MID 1, FWD 0
Goal conceded (GK/DEF, after the first)	−1 each
Penalty save (GK, shootouts excluded)	+3
Saves (GK)	+1 per 3 saves
Tackles (MID)	+1 per 3 tackles
Chances created (MID)	+1 per 2 chances
Shots on target (FWD)	+1 per 2 on target
Yellow / red card	−1 / −2
Own goal	−2
Penalty won / conceded	+2 / −1
Direct free-kick goal	+1 bonus
Scouting bonus	+2 if > 4 points while owned by $< 5\%$ of teams

The scouting bonus is the lever for differential picks: a sub-5%-owned player who scores more than four points in a match earns an extra 2, cheap upside the player-points model prices in as an expected value (`rules.py:179`).

Chapter 5

Models

The models turn the collected data into two things the optimizer needs: how far each national team is expected to go, and how many points each player is expected to score. Three modules do the work: `elo.py`, `progression.py` and `player_points.py`, with `calibrate.py` as the offline refit.

5.1 Elo ratings and the outcome model

One pass over the historical results produces a current Elo rating per team and a training set for a match-outcome model (`elo.py:1`).

5.1.1 Elo update

The expected score for a rating gap $\Delta = R_{\text{home}} - R_{\text{away}}$ (plus a home-advantage term of 80 when the match is not on neutral ground) is the logistic

$$E = \frac{1}{1 + 10^{-\Delta/400}}. \tag{5.1}$$

After a match with actual result $S \in \{1, 0.5, 0\}$ the ratings move by $\delta = K\,(S - E)$, symmetric between the two teams. The update factor K scales with the importance of the match (60 for a World Cup, 40 for a qualifier or continental cup, 20 for a friendly) and with the goal margin: K is multiplied by 1.5 for a two-goal win and by $1.75 + (m - 3)/8$ for a margin $m \geq 3$ (`elo.py:33`).

5.1.2 Outcome model

A multinomial logistic regression maps the rating gap to win, draw and loss probabilities. It trains on competitive matches since 1998, excluding friendlies, with the rating gap as the single feature and the three-way result as the label (`elo.py:82`). The 2026 results enter the ratings from the live FIFA feed, not the dataset, so the ratings stay current to the last played match.

```
if row.date >= TRAIN_FROM and row.tournament != "Friendly":
    X.append([diff])
    y.append(1 if actual == 1.0 else 0 if actual == 0.5 else -1)
model = LogisticRegression(max_iter=1000)
model.fit(np.array(X), np.array(y))
```

Listing 5.1: The outcome-model fit (excerpt).

5.2 Monte Carlo progression

`progression.py` simulates the whole tournament to estimate $P(\text{team still playing at round } r)$ for every squad. The default is 2000 simulations.

5.2.1 The simulation

Each simulation plays the remaining group fixtures (completed ones enter with their real result), ranks the groups, allocates the eight best third-placed teams with FIFA's official slot table, and runs the R32-to-final bracket. The bracket template and the third-place allocation were extracted from the game's own bracket-predictor JavaScript and live in `data/bracket_2026.json`. A knockout tie that is level after ninety minutes is decided by an Elo-weighted coin for extra time and penalties (`progression.py:301`).

Two performance and correctness choices stand out. Per-fixture probabilities come from a manual softmax over the fitted logistic coefficients, since calling `predict_proba` the roughly hundred thousand times the loop needs would be far too slow (`progression.py:92`). And a real knockout result from the feed overrides a simulated pairing by team pair in either orientation, never by position, so a half-played knockout round never pins a real score onto the wrong simulated tie (`progression.py:142`).

5.2.2 Market calibration

The ratings carry the economic adjustment from Section 3.5. Where the market quotes a fixture, the head-to-head probability replaces the model's. The outright winner odds calibrate the ratings in a second pass: each quoted team's rating is nudged by

$$\Delta R_{\text{sid}} = \text{clip}\left(G \cdot \ln \frac{p_{\text{market}}}{p_{\text{sim}}},\ -M,\ +M \right), \tag{5.2}$$

with gain $G = 60$ and clamp $M = 80$, so the market corrects the model rather than replacing it (`progression.py:227`). The simulation then re-runs with the nudged ratings. If any data source breaks, the whole simulation falls back to a price-based survival heuristic so the tool keeps working offline (`progression.py:73`).

5.2.3 Two horizon outputs

The survival curve yields two distinct signals. The sum of $P(\text{still playing})$ over the rounds from the next one onward is the horizon weight: how many more rounds a player's team is expected to play. The probability of reaching the round after next is $P(\text{advance one round})$, which is what the Qualification booster pays on (`progression.py:403`). They are kept separate on purpose; conflating "expected to go deep" with "wins the next match" only coincides at the semi-final.

5.3 Expected points per player

`player_points.py` blends the priors, scales them by the chance the player plays, blends in observed form, and weights the horizon. The signal priority is: the club projection where a confident name match exists, the price heuristic otherwise, and observed World Cup form taking over as minutes accumulate (`player_points.py:1`).

5.3.1 The club projection

`project_club_ep` runs each player's club season totals through the official scoring table to a per-match expected-points figure. Per-90 rates are scaled by the player's minutes share, position-specific terms are added (saves and goals conceded for keepers, tackles and chances for midfielders, shots on target for forwards), and a clean-sheet term uses the keeper's own rate or a flat league-typical rate for outfield players, since FBref has no outfield clean-sheet data (`player_points.py:182`).

5.3.2 The club-price blend

The club projection runs hotter than the price prior, a different scoring environment with generous proxies, and only Big-5 players have one. It is shrunk halfway toward the price prior, so the scales stay comparable while the club ordering survives. The blend weight w defaults to 0.5 and is calibratable (`player_points.py:244`):

$$\text{prior} = w \cdot \text{club} + (1 - w) \cdot \text{price}, \qquad \text{fall back to price where no club match exists.} \tag{5.3}$$

5.3.3 Two separate P(plays) factors

The model multiplies expected points by the chance the player plays, and it keeps that chance in two parts, applied at two points on purpose (`player_points.py:244`).

Depth chart scales the *prior*. This is the chance the player starts for his national team, built strongest-signal-last: the price rank within (squad, position) is a weak pre-tournament prior; observed starts override it wherever they exist; a manual `starter_overrides.json` entry wins over both (`player_points.py:129`).

Availability scales the *final* expected points, form included. An injury or a confirmed benching is fresh news that past form must not override, so a ruled-out player is worth about zero no matter his form (`player_points.py:277`).

The price rank is capped so a nailed rank-1 starter reads about 0.93, not 1.0: no starter is ever a literal certainty given late knocks, rotation and suspensions (`player_points.py:114`).

5.3.4 Form blend, opponent factor and scouting EV

Observed World Cup form takes over the quality side with a weight proportional to matches played, full trust after four matches:

$$\text{ep} = (1 - b) \cdot \text{prior} + b \cdot \text{form}, \qquad b = \frac{\min(\text{matches}, 4)}{4}. \tag{5.4}$$

The next-round figure is scaled by an opponent factor from the Elo expected score against that round's opponent, ranging from about 0.75 against a far stronger side to about 1.25 against a weaker one (`player_points.py:284`). The scouting bonus enters as an expected value: an increasing, capped proxy for $P(> 4 \text{ points})$ times the 2-point bonus, applied only to sub-5%-owned players (`player_points.py:271`).

`expected_value` assembles the columns the optimizer reads: `ep_match` (next round, opponent factor applied), `future_rounds`, `p_advance`, `ep_horizon` (base points times expected

future rounds) and `p_play` (depth chart times availability, surfaced in the report) (`player_points.py:317`).

5.4 Calibration

`calibrate.py` refits the priors against real results at the end of a round. It reads the latest local feeds (no network), compares the pre-form prior against the points players have actually scored, and proposes new weights. It is a dry run by default; `-write` persists the proposal to `data/state/calibration.json`, which `player_points.apply_calibration` layers over the built-in defaults at runtime (`calibrate.py:1`).

5.4.1 The observed target and the fits

The target is observed points per appearance for players with at least one match (`calibrate.py:61`). Two weights are fit, a deliberately small and stable set since two rounds is little data: the price slope and intercept by least squares of observed points on price, and the club-price blend weight by a grid search ranking correlation first and then error.

5.4.2 The improvement gate

A candidate weight enters the proposal only when it beats the current default on the metrics that drive squad choice. Ranking (correlation) leads because the optimizer orders players; mean absolute error is the secondary check. A candidate must improve at least one metric beyond its epsilon while worsening neither (`calibrate.py:119`).

```
no_worse = rc >= bc - corr_eps and rm <= bm + mae_eps
better = rc > bc + corr_eps or rm < bm - mae_eps
return no_worse and better
```

Listing 5.2: The is_improvement gate.

The gate rejects two failure modes: a least-squares price fit that lowers the average bias while raising the typical error on the right-skewed points distribution, and an error-only blend search that collapses to a noise-driven corner early in the tournament. The proposal stays empty until a refit genuinely helps, and writing also refuses below thirty observations (`calibrate.py:51`).

Chapter 6

Optimization

The optimizer turns expected points into decisions: which fifteen to own, which eleven to start, who to captain, which transfers to make, and which boosters to play. `optimizer/squad.py` solves the squad as an integer program; `optimizer/transfers.py` handles the round-to-round changes; `optimizer/captain.py` handles the captain, the bench order, the booster advice and the timing option values.

6.1 Squad selection as an ILP

`build_squad` picks the optimal fifteen, the starting eleven and the captain with PuLP over the bundled CBC solver. Each player carries three binary variables: in the squad, in the starting eleven, captain (`squad.py:83`).

6.1.1 Objective

A starter earns a per-player score that blends the next round with a discounted horizon, the captain earns the next-round figure again (the double), and a benched player counts at a small weight:

$$\text{score}_i = \text{ep_match}_i + \lambda\,(\text{ep_horizon}_i - \text{ep_match}_i), \qquad \lambda = 0.35, \tag{6.1}$$

so the next round dominates while likely finalists still get pulled in early (`squad.py:36`). The objective is

$$\max \sum_i \Big[\text{starter}_i\,\text{score}_i + \text{captain}_i\,\text{ep_match}_i + (\text{in_squad}_i - \text{starter}_i)\,\text{bench_value}_i\Big]. \tag{6.2}$$

6.1.2 Constraints

The budget, squad size, positional composition, per-nation cap, eleven starters, the formation bounds on starting defenders, midfielders and forwards, one goalkeeper starting, and exactly one captain who must be a starter (`squad.py:114`). In transfer mode an extra constraint keeps all but K of the current squad:

$$\sum_{i \in \text{kept}} \text{in_squad}_i \;\geq\; 15 - K. \tag{6.3}$$

6.1.3 Bench weight and the backup goalkeeper

An outfield substitute counts at a flat bench weight of 0.15. The backup goalkeeper is special: you start exactly one keeper, and a keeper is auto-subbed only by the other keeper, so the second one scores only when your number-one plays zero minutes and no manual change is made, which is rare for a nailed starter. He gets a much smaller weight (0.03), scaled by his own $P(\text{plays})$ so a keeper who has lost his starting spot is not credited bench value through his team's deep run, plus a microscopic price tiebreak so the budget is not parked on an expensive bench keeper who never plays (`squad.py:56`).

```
base = _player_score(row)
if row.position != "GK":
    return BENCH_WEIGHT * base
p_play = row.p_play if "p_play" in row.index else 1.0
return BENCH_GK_WEIGHT * base * float(p_play) - BENCH_GK_PRICE_TIEBREAK * row.price
```

Listing 6.1: Backup-GK bench value.

The solver stays on the bundled CBC for portability rather than the external `cbc` binary PuLP 4.0 prefers, and the migration warning is muted (`squad.py:149`).

6.2 Transfers

`plan_transfers` allocates the round's changes under the rules and the user's strategy (`transfers.py:96`). The free allocation comes from the round plus any group-stage carryover. An unlimited window (the R32) re-solves the squad from scratch, every change free.

6.2.1 The paid-transfer rule

The default is to stay within the free allowance. A voluntary paid transfer, at -3 points, is taken only when it pays for itself on a conservative next-round basis. The same swap would be free at the next window, so paying now buys only this round's upgrade; an extra paid transfer is accepted only when it lifts the squad's next-round points (the starting eleven plus the captain's double) by more than the penalty (`transfers.py:84`). Horizon value is excluded from this test on purpose, since you can capture almost all of it for free next window. This keeps paid transfers rare and leaves the Wildcard as the route for a big rebuild.

6.2.2 Forced versus voluntary

Forced moves are separate. When more current players are unplayable (eliminated or transferred, so their `ep_match` is zero and they drop from the pool) than the allowance can shed, the planner walks the change count up from the free allowance until `build_squad` returns a valid squad: that is the forced baseline (`transfers.py:123`). It then weighs voluntary paid transfers on top of the baseline by comparing penalty-adjusted next-round scores. The report flags the two kinds of -3 hit distinctly, and if even the deepest level cannot keep enough of the squad, a free full rebuild is the only valid plan, a Wildcard round in practice.

6.3 Captain, vice and bench order

The ILP picks the captain. The vice-captain is the next-best starter by expected points, and the bench order follows expected points, since that order drives the auto-subs and all bench

minutes are known by round end (`captain.py:26`). The booster advice applies the official effects plus the user's strategy, with one guard: the single booster to activate this round is chosen up front by priority (Qualification at the semi-final, then a late-knockout Wildcard, then Maximum Captain) so a lower-priority booster can never preempt a planned higher-priority one (`captain.py:356`).

6.4 Option value: the timing decisions

Two decisions exploit the live-edit rules. The captain is switchable live, and a bench player can be subbed in live only while he has not yet kicked off. Both turn kickoff order into value.

6.4.1 Option-value captaincy

A single-match captain score is volatile, and the armband can be moved from a completed player to one not yet played, unlimited times. Captaining the earliest strong starter first and rolling the armband to a later, higher-expected one if he disappoints banks the better of the early realised score and the later expectation, worth more than captaining the top-expected starter outright, even at a meaningful gap, because the expected-points estimate is itself noisy (`captain.py:157`).

The value is a backward recursion. Holding the armband on the first player, then after each match keeping it or rolling to the next not-yet-played player, gives

$$V_i = \mathbb{E}\big[\max(X_i, V_{i+1})\big], \qquad V_{\text{last}} = \mu_{\text{last}}, \tag{6.4}$$

where $X_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ is the captain score of the i-th player in kickoff order. With a normal X, $\mathbb{E}[\max(X, f)]$ has the call-option closed form

$$\mathbb{E}[\max(X, f)] = \mu\,\Phi(d) + f\,\Phi(-d) + \sigma\,\phi(d), \qquad d = \frac{\mu - f}{\sigma}, \tag{6.5}$$

with Φ and ϕ the standard normal CDF and PDF (`captain.py:195`). The score's standard deviation is a fraction of its expected value with a floor, so low-expected players still carry spread. The recursion is monotone: starting earliest is never worse than captaining any single player outright.

`plan_captain` returns the highest-expected starter (the anchor) unless captaining an earlier strong starter first and rolling to the anchor is worth more: the doubled option premium must beat the one-off live-switch cost (the auto-sub and vice insurance forfeited by the first manual move). A floor on the candidate set keeps weak starters out, replacing an earlier hard tolerance gate that rejected the play (`captain.py:224`).

```
v_seq = sequential_captain_value(mus, cv=cv, sd_floor=sd_floor)
v_rest = sequential_captain_value(mus[1:], cv=cv, sd_floor=sd_floor)
sd0 = max(cv * mus[0], sd_floor)
p_roll = _norm_cdf((v_rest - mus[0]) / sd0) if sd0 > 0 else float(mus[0] < v_rest)
premium = 2.0 * (v_seq - top_ep) - p_roll * switch_cost
if premium > min_premium:
    return CaptainPlan(initial=strong[0][0], anchor=anchor_id, ...)
```

Listing 6.2: The captaincy premium decision.

6.4.2 Temporal option value of the eleven

The same logic decides the start-bench split. A bench player can be brought on live only while he has not kicked off, so a strong early starter belongs in the eleven (his points bank and you cannot react to him on the bench anyway) and the bench is better kept for late players, where the manual-sub option survives. `retime_lineup` re-picks the eleven within the ILP's formation, swapping only within a position so the lineup stays legal, ranking by a start premium (`captain.py:84`):

$$\text{premium}_i = \text{ep_match}_i \cdot \big(1 - \text{BENCH_WEIGHT} - w_{\text{opt}} \cdot \text{reactivity}_i\big), \tag{6.6}$$

where reactivity is the fraction of squad-mates who kick off before player i, how usable he is as a live substitute, and $w_{\text{opt}} = 0.10$. Expected points stay primary: the timing term only flips genuinely close same-position calls.

The split is scored on this round's `ep_match`, not the horizon-weighted squad score the ILP uses. Who you own is a horizon bet, decided in squad selection; who you start tonight earns only this round's points, since benching a player does not forfeit his future rounds. Using the horizon here had wrongly benched in-form players from weaker nations, whose shorter expected run dragged their score down (`captain.py:47`). The goalkeeper split is left untouched, since a keeper is auto-subbed only by the other keeper and timing cannot help.

Chapter 7

Orchestration

Two commands drive the engine. `advise` plans the next round between rounds; `live` advises during a round. Both refresh the feeds first, then assemble a structured result the CLI prints and the web layer serves.

7.1 advise

`compute_advice` returns an `AdvicePlan`, a dataclass holding everything needed to print the plan and serve it as JSON, with no console formatting baked in (`advise.py:28`). `run` is a thin print wrapper over it. The plan carries the round, budget, country cap, mode, the squad rows, captain and vice, the timing notes, the bench order, the transfer summary, the move list, the availability watch, the unverified-starter warnings, the booster advice and the fixture schedule.

7.1.1 Picking the round to plan

`next_round_number` returns the first round still open for edits. A done round is over, and a round being played is locked at its first kickoff, so both are skipped: a transfer confirmed now applies from the next open round (`advise.py:115`).

7.1.2 Apply-aware planning

The committed-squad case needs care. When the open round's squad is already applied (the user ran `team apply`) and the round is not yet locked, the free transfers for that window are spent, so re-offering them would be wrong. `plan_round_after_apply` resolves this (`advise.py:133`):

- if owned players became unplayable, stay on the open round and propose only the forced replacements, each now costing −3, with the allowance pinned to zero (mode `forced`);
- otherwise there is nothing left to do, so preview the next round (mode `preview_next`, not applyable, not saved), or report nothing more to do if this was the last round.

The mode flows through to both the CLI and the web UI through `applied_state` and `applied_note`. A preview firms up once the open round locks.

7.1.3 Assembling the move list

After the squad or transfer plan is solved, `compute_advice` runs the timing logic in order: `retime_lineup` re-picks the eleven, then `recommend_captain` sets the initial captain, then the vice, bench order and notes follow from the solved squad (`advise.py:563`). It then collects the availability watch (players below 75% to play), the unverified-starter warnings (starters known only from the price guess, with goalkeepers always named), the booster advice and the fixture schedule in Italian local time, which tells the user when to re-run `live`. An applyable plan is written to `last_advice.json`; a preview or a completed final is not, so it never overwrites the saved squad (`advise.py:649`).

7.1.4 The from-scratch comparison

`advise -from-scratch` builds the unconstrained ideal fifteen, ignoring the saved team and the transfer limits, and diffs it against the current squad: the overlap, the upgrades it would bring in, the players it would drop and the per-round points gap (`advise.py:268`). It is a target to steer toward over several windows or a Wildcard, never written to `last_advice.json`, so `team apply` is not offered for it.

7.2 live

`compute_live_state` answers two questions against partial results, both priced in expected value (`live.py:1`). If the captain has finished, is there an unlocked starter whose expected points beat the captain's banked score? If a starter has finished, is there an unlocked bench player expected to beat his score, formation permitting?

7.2.1 The first-change cost

Every suggestion prices the official catch: the first manual change of the round cancels auto-subs and the vice insurance. `first_change_cost` estimates the forfeited auto-sub insurance as the number of still-to-play starters times a rough did-not-play probability times the mean bench expected points (`live.py:189`). After the first change that cost is sunk and later changes are free, which the advice states.

7.2.2 Captain before he has played

Before the captain has played, `live` surfaces the option-value plan from Section 6.4 proactively: if a strong starter plays earlier, it advises captaining him first and rolling the armband toward the higher-expected anchor if he disappoints, each suggestion carrying its deadline, the incoming player's kickoff in Italian time (`live.py:212`). Confirmed lineups override the injury factor in the availability used here.

7.2.3 The round outlook

`live` ends with one line per owned player still to play this round, showing pure $P(\text{plays})$, the depth chart times availability. Expected points are decided in the lineup, transfer and substitution logic, not here; the outlook only flags who might not take the pitch (`live.py:414`).

7.2.4 Recording an executed change

`live` only suggests; it cannot see what the user did in the game, so the saved eleven would go stale and every re-run would re-suggest the same move. The recording flags fix this: `-captain`, `-sub-out` with `-sub-in`, and `-vice` record the real change into `my_team.json`, so the advice reflects it and stops looping (`live.py:576`). Names resolve within the saved squad, and a substitution is validated for squad membership and a legal formation before anything mutates. Any recorded change also marks the round as changed, since the first manual change is what cancels auto-subs and the vice insurance. Bare `-mark-changed` only flags the round.

7.2.5 The apply-over-live guard

Recording a future-round plan while an earlier round is still being played would overwrite `my_team.json` and make `live` read the wrong eleven for the round on the pitch. `team apply` blocks this by default, comparing the live round to the plan's round, and `-force` overrides (`advise.py:755`, `live.py:454`).

Chapter 8

State and the semi-automatic flow

The game's write API is authenticated and out of scope, so the advisor cannot read the team back from FIFA. State that the game would hold lives in two local JSON files instead, and the user keeps them honest by confirming what was done.

8.1 The two files

`data/state/my_team.json` is the source of truth for the current squad, loaded into a `TeamState` dataclass (`state.py:15`). It holds the fifteen player ids, the eleven starters, the captain and vice, the ordered outfield bench and the backup keeper, the bank, the boosters used, the last round the team was set for, any carried-over transfer, and the round of the last manual live change.

`data/state/last_advice.json` is the plan `advise` just produced. Keeping the two separate means `advise` can compute and print a plan without touching the committed squad; only `team apply` promotes the plan to the truth.

`load` tolerates old schemas: it ignores any key not in the current dataclass, so a stale `my_team.json` still loads instead of raising (`state.py:33`).

```
known = {f.name for f in fields(TeamState)}
return TeamState(**{k: v for k, v in data.items() if k in known})
```

Listing 8.1: Schema-tolerant load.

8.2 The commands that write state

`team apply` reads `last_advice.json` and writes it into `my_team.json`, after the apply-over-live guard from Section 7.2 clears (`advise.py:755`). This is the step the user runs once the moves are typed into the game.

`team use-booster <name>` appends a booster to `boosters_used`, so the advisor stops suggesting it and the one-per-tournament rule holds. The advisor cannot observe an in-game activation, and most boosters are reversible until lockout, so the user records it here. The name is normalized and the legacy `mystery` maps to `clean_sheet_shield` (`advise.py:829`).

`live -captain / -sub-out / -sub-in / -vice` record an executed in-game change directly into `my_team.json`, as Section 7.2 covers.

`team show` prints the current squad with its role marks, the carried-over transfers and the boosters used (`advise.py:876`).

8.3 Why state lives locally

The advisor runs a semi-automatic loop: it reads public data, computes a plan, and prints instructions; the user executes them and confirms. Nothing crosses into the FIFA account. The local files are the only memory of what the user actually did, so they must mirror the in-game squad for the next round's plan to be correct. The guards exist for exactly this reason: the schema-tolerant load survives a version bump, the apply-over-live block stops a future plan from clobbering a live round's eleven, and the recording flags stop `live` from re-suggesting a change the user already made.

Appendix A

Lessons learned

These are the constraints and traps the code comments record, the ones worth not rediscovering. They are grouped by area.

A.1 FIFA feeds

- The fantasy CDN returns 403 without a browser `User-Agent`. Paths under `json/fantasy/` other than the four known feeds also return 403; do not keep probing them.
- Appearance points are cumulative: 1 for any minutes, 2 from sixty. The live status strings are `complete` / `playing` / `scheduled`, not `completed`. `roundPoints` is a dict keyed by the round number as a string.
- The fantasy feed has no injury data: `status` is only `playing` or `transferred`. At matchday 1 both an injured-doubt and a nailed starter read `playing`.

A.2 FIFA Match Centre

- Competition 17, season 285023 for the men's 2026 World Cup, both verified against the live feed.
- Card codes: 1 single yellow, 2 sending-off (verified on MEX-RSA, where Montes, Zwane and Sithole each carried a lone Card 2). The straight-red code is assumed to be 3 and included for safety.
- Only cache a match detail that carries a lineup, and treat a cached but lineup-less payload as a miss. An empty pre-lineup fetch cached for a since-completed match poisons every later read; this both prevents and self-heals it.
- The team bridge is exact (`abbr` equals `IdCountry`); the player bridge is fuzzy within the squad, which rejects a wrong same-surname match.

A.3 The Odds API

- Restrict the sport key to the `soccer_fifa_world_cup` family. The cricket, rugby, club, women's and qualifier world cups all share the `world_cup` substring, and the cricket key was being picked for the head-to-head market, returning zero soccer fixtures.

- Three regions cost three credits per call, worth it for coverage against the 500-per-month free quota. De-overround per bookmaker before averaging.

A.4 Club stats (FBref)

- This `soccerdata` version exposes only the standard, keeper, shooting, playing-time and misc tables, no passing or defense, and the 25/26 Big-5 pages carry no expected-goals columns. Tackles come from misc tackles-won; chances-created is proxied from assists at about seven key passes each.
- Collapse a mid-season mover's rows before merging the player-summed extra tables, or the totals double-count.
- The first fetch downloads a chromedriver for Cloudflare and takes minutes. Cache the fuzzy name match, keyed by an input hash: it dominated an `advise` run.

A.5 Availability

- The Wikipedia parser must flag only the player named before a withdrawal verb with an injury noun in the same clause, never the "replaced by" healthy replacement. That trap once produced 37 false positives. A defensive cap discards the pass if the page format drifts.
- API-Football's free plan has no access to season 2026: `/injuries` returns zero with a plan error. The path stays inert until a paid key.
- Every source returns an empty result on any failure, so availability degrades cleanly to the price depth chart alone.

A.6 Models and optimizer

- The price rank is a weak depth-chart prior, near-flat for keepers: it ranked the wrong number-one for Spain and Egypt at matchday 1. Observed starts override it; a manual override file wins over both. Cap the rank-1 reading at about 0.93, never a certainty.
- Keep the two $P(\text{plays})$ factors separate: the depth chart scales the prior, the availability factor scales the final expected points, because an injury must not be overridden by past form.
- In the Monte Carlo, resolve a real knockout result by team pair, not by position, so a half-played round never pins a score onto the wrong simulated tie. Use a manual softmax in the loop; `predict_proba` is far too slow.
- Score the start-bench split on this round's `ep_match`, not the horizon-weighted squad score: benching a player tonight does not forfeit his future rounds. The horizon belongs in squad selection.
- Keep paid transfers rare: judge a voluntary -3 on the next round alone, since the same swap is free next window. The Wildcard is the big-rebuild route.

A.7 Calibration

- Gate every refit on correlation first, then error. A least-squares price fit can lower bias while raising typical error on the right-skewed points distribution, and an error-only blend search collapses to a noise-driven corner early. Propose only what beats the default, and refuse to write below thirty observations.